



CHICAGO NOV 15-20

Gregory Bolet

My Submissions

Make a New Submission

My Conflicts

My Expertise

My To-Do List i

hide

You have 2 pending tasks. These can be viewed [here](#).

All Reviews of pap899 details

Submission: Off-the-Shelf LLMs are Static Roofline Triage Tools for GPU Kernels

Contributors: Bolet, Cameron, Georgakoudis, Hasabnis, Menon, Oren, Parasyris

Sections Below

Full Reviews · [jump down](#)

Authors' Rebuttal Of Reviews · [jump down](#)

Committee Comments · [jump down](#)

Full Reviews

Reviewer 1

[↑ top](#)

► Summary and High Level Discussion

monospace

In this paper, whether off-the-shelf LLMs can help predict the profiled arithmetic intensity well enough to support early GPU-kernel triage without direct hardware access is investigated. 254 real-world CUDA and OpenMP GPU kernels from the HeCBench suite are evaluated across four different NVIDIA GPU architectures, which lead to four findings: 1) Prompts that include post-compilation information substantially improve half-precision RAI estimates and compute/bandwidth-bound classifications for the stronger models. 2) The strongest overall results come from the more capable models, but a much cheaper model remains competitive when only pre-compilation information is available. 3) Prediction quality is not uniform: larger-cache GPUs are easier settings than smaller-cache GPUs, and prediction errors concentrate around code patterns such as division, transcendental math functions, reusable subexpressions, and loop-invariant floating-point work.

► Strengths

markdown monospace

1. The idea of using LLMs to predict the profiled arithmetic intensity of GPU kernels is interesting.
2. A wide range of real-world CUDA and OpenMP GPU kernels are used in the evaluation.

► Weaknesses

markdown monospace

1. The prediction errors are high.

2. The cost of using the SOTA LLMs such as GPT 5.4 might be higher than launching a GPU VM instance in the cloud and measuring the real performance.

► Comments for Rebuttal

markdown monospace

1. Given the high prediction errors, how can the proposed method be used in practice?
2. Calling the API of SOTA LLMs can be quite expensive. With the same cost, why don't you launch a GPU VM instance in the cloud and measure the real performance? It would be interesting to find out which one is cheaper.

► Detailed Comments for Authors

monospace

The idea of using LLMs to predict the profiled arithmetic intensity of GPU kernels is very interesting. However, the evaluation do not show very promising results. The prediction errors are too high. As listed in Table III, even with the kernel-relevant SASS sections are included in the prompts, none of the three LLMs is able to achieve prediction errors below 10% for more than 50% of the cases. The table also lists percentage of cases whose prediction errors are below 50%. But frankly speaking, if the prediction error is 50%, this method is not going to be useful in practice. Moreover, calling the API of SOTA LLMs can be quite expensive. With the same cost, why don't you launch a GPU VM instance in the cloud and measure the real performance?

Reviewer 2

[↑ top](#)

► Summary and High Level Discussion

monospace

The paper evaluates the ability of general-purpose chatbot-style LLMs to predict the arithmetic intensity of various GPU kernels in CUDA and OpenMP taken from the HeCBench benchmarks across four GPU architectures. For each kernel/GPU combination, the authors collected ground truth profiling with NVIDIA Nsight Compute as well as the SASS. For each kernel/GPU combination, and for three LLMs, the LLM was given a prompt incorporating the kernel source, target hardware, and various metadata, along with (optionally) the SASS, requesting it to predict arithmetic-intensity-relevant parameters. The authors evaluate the error in predicted RAI compared to the profiled ground truth, both where the LLM was and was not provided with SASS, for FP16, 32, and 64. They then analyze how well the LLMs were able to categorize kernels into compute- or bandwidth-bound classes.

► Strengths

markdown monospace

- Large number of kernels evaluated from benchmark (254 kernels, of which 155 were CUDA and 99 OpenMP device offload)
- Predicted RAI evaluated across many parameters (3 LLMs, two runtimes, source only vs source+SASS, GPU model, FP16/32/64)
- Example prompt, input, and output provided aids in understanding the technique
- Will publicly release benchmark

► Weaknesses

markdown monospace

- Each combination run through prediction only once; consistency of prediction for a given combination cannot be evaluated
- Results not compared against existing techniques (e.g., performance-specific models, static analysis, etc.)

► Comments for Rebuttal

monospace

As you mention in your related work section, "Long before LLMs, prior work studied GPU performance prediction using analytical and learned models built from program structure, microarchitectural assumptions, or low-level code features." However, the paper studies the use of off-the-shelf LLMs for roofline prediction in isolation. How does the technique you use in the paper compare to the prior state of the art? For example, how does the LLM approach compare to older models/static analysis tools at classifying kernels as compute vs. bandwidth bound?

► Detailed Comments for Authors

monospace

You note that "Due to the expensive query costs associated with the GPT-5.4 and Opus-4.6 LLMs, we only perform RAI predictions once for each kernel across all four target GPUs." Given the mentioned costs, this is understandable, but given the inherent nondeterminism of LLMs, it might be a good idea to pick at least a few representative examples and run them through RAI prediction repeatedly in order to evaluate how consistent the predictions are.

You note your code-voting feature where you provide the code repeatedly to several different low-cost models with instructions to identify whether certain features are present that can cause failures in RAI prediction. I wonder how this technique compares to unconditionally including in the system prompt a list of these features and how they might skew the results, so that the main model can take these into account.

Providing a list of research questions in the introduction is helpful; however, you don't really come back to them explicitly. I would recommend restating, and explicitly answering, the research questions in the conclusion.

Reviewer 3

[↑ top](#)

► Summary and High Level Discussion

markdown monospace

This paper investigates whether off-the-shelf Large Language Models (LLMs) can be utilized without task-specific fine-tuning or training to act as early-stage hardware-free performance triage tools. Typically, optimizing GPU kernels relies heavily on physical hardware profiling to determine if a kernel is compute-bound or bandwidth-bound (Roofline analysis). However, hardware access is often restricted or unavailable during early stages of software development. To overcome this bottleneck, the authors evaluate whether LLMs can analyze code representations statically and predict Roofline Arithmetic Intensity (RAI).

- **Decomposed Prediction Task:** Rather than asking the models to guess the RAI directly, the authors break down the prompt requirement into seven underlying metrics: grid size, block size, FP16/FP32/FP64 operations, and DRAM bytes read/written.
- **Evaluation Framework:** The authors build and profile 254 real-world CUDA and OpenMP kernels from the HeCBench suite across four separate NVIDIA GPU architectures (RTX 3080, A10, A100, and H100).

Prompting Variations: They test two primary prompting setups: Source-only (pre-compilation information) and Source+SASS (which appends post-compilation GPU assembly instructions). They evaluate three models:

Claude Opus 4.6, GPT-5.4, and a budget open-source baseline (GPT-OSS).

► Strengths

monospace

This paper shows an interesting way of performance prediction.

► Weaknesses

markdown

monospace

- This is an oversimplification of the performance modeling problem. For example, it focuses on the first-invocation. To make the prediction task manageable for the LLMs, the authors restricted their profiling evaluation exclusively to the first invocation of each kernel within an execution run. This intentionally avoids forcing the models to reason about inter-invocation cache reuse and cross-kernel temporal effects. In real-world multi-kernel pipelines or iterative application lifecycles, data residing in the L2 cache from previous operations profoundly affects DRAM traffic. By narrowing the scope only to first-time cold-start executions, the framework fails to accurately represent practical application workloads where cache-resident data limits the utility of a purely static DRAM-level prediction.

► Comments for Rebuttal

monospace

See above.

► Detailed Comments for Authors

monospace

See above.

Reviewer 4

[↑ top](#)

► Summary and High Level Discussion

monospace

The paper asks the question if frontier model LLMs can predict the Roofline Arithmetic intensity of GPU kernels without any GPU runs. The paper then tests this hypothesis with three frontier models (2 closed source and one open-source). The results, as can be seen in Figure 6, is that LLMs are, to a large extent, very bad predictors of RAI.

► Strengths

markdown

monospace

- The paper shows that LLMs are bad predictors for RAI even when given plenty of data

► Weaknesses

markdown

monospace

- I am not entirely sure about the utility of the framework. First of all, the costs of doing the tests using LLMs can probably fast approach the price of actually doing the measurements. For example, the Opus 4.6 costs would have been sufficient to do the analysis on all kernels using real measurements.

- To put it lightly, the errors from the LLMs are very large

► Comments for Rebuttal

markdown monospace

- Can you please explain why would anyone use the tool to predict the RAI given the wildly expensive costs and almost complete randomness of the output?

► Detailed Comments for Authors

monospace

Thank you for submitting your work to SC!

Unless I am missing something major, your results actually show that the RAI predictions are completely off. Having the median around zero, with both positive and negative errors possible, some of which are orders of magnitude wrong, makes it very hard to believe/use the predictions of the LLMs. Typically, in production, we are after even 5% improvements in the kernel performance. Having the errors of the predictions higher than this means that the predictor really has no use for kernel programmers and engineers. I am not entirely sure how this tool/result is any useful.

While I appreciate the effort put in curating the dataset, I am not entirely sure what the core contribution here is. If it is the dataset, then it is really not well described in the paper. If it is the evaluations of LLMs, I am not entirely sure how useful is that since it is a very limited test with many limitations.

Authors' Rebuttal Of Reviews

[↑ top](#)

[submit rebuttal](#)

Committee Comments

[↑ top](#)

Committee Comments for Authors

None